```
colon → (whitespace | comment)* ":";
comma → (whitespace | comment)* ",";
dot → (whitespace | comment)* ".";
doublevbar → (whitespace | comment)* "||";
equalSign → (whitespace | comment)* "=";
eventualSendOperator → (whitespace | comment)* "< − :pound → "#";
assignSlot → (whitespace | comment)* "::=";
hat → (whitespace | comment)* "^";
inverseMixinOperator → (whitespace | comment)* ":>pound → "#";
lbracket → (whitespace | comment)* "[";
lcurly → (whitespace | comment)* "{";
lparen → (whitespace | comment)* "(";
langleBracket → (whitespace | comment)* "<";
mixinOperator → (whitespace | comment)* "<:pound → "#";
angleBracket → (whitespace | comment)* ">";
rbracket → (whitespace | comment)* "]";
rcurly → (whitespace | comment)* "}";
rparen → (whitespace | comment)* ")";
semicolon → ";";
slash → (whitespace | comment)* "/";
vbar → (whitespace | comment)* "|";
letter → "a" - "z" | "A" - "Z";
specialCharacter → "+" | "/" | "\'" | "*" | "~" |
                   "<" | ">" | "=" | "@" | "%" |
                   "|" | "&" | "?" | "!" | "'";
underscore → "_";
id → (letter | underscore) (letter | digit | underscore)*;
identifier → (whitespace | comment)* id;
unarySelector → identifier;
kw → id ":";
kws → kw+;
keyword → (whitespace | comment)* kw;
setterKw → kw ":";
setterKeyword → (whitespace | comment)* setterKw;
beginComment → "(*";
endComment → "*)";
metadataTag → ":" id ":";
comment → beginComment [metadataTag] (( endComment any) | comment)*
endComment;
binSel → (specialCharacter | "-") (specialCharacter*);
binarySelector → (whitespace | comment)* binSel;
digit → "0" - "9";
digits → digit+;
uppercaseLetter → "A" - "Z";
extendedDigits → (digit | uppercaseLetter)+;
radix → digits "r";
```

```
fraction → dot digits;
extendedFraction → dot extendedDigits;
exponent → "e" ["-"] digits;
decimalNum → ["-"] digits [fraction] [exponent];
radixNum → radix,
            ["-"]
           extendedDigits
           [extendedFraction]
           [exponent];
num → radixNum | decimalNum;
number → (whitespace | comment)* num;
character → digit | letter | specialCharacter | "[" | "]" |
             "{" | "}" | "(" | ")" |
             "^" | ";" | "$" | "#" |
             ":" | "." | "-" | underscore | "'";
twoDblQuotes → "'" "'";
aChar → "'" (character | twoDblQuotes | "`" | ' ') "'";
characterConstant → (whitespace | comment)* aChar;
twoQuotes → "`" "`";
stringBody → (character | aWhitespaceChar | "'" | twoQuotes)*;
str → "`" , stringBody "`";
string → (whitespace | comment)* str;
sym → str | kws | binSel | id;
symbol → (whitespace | comment)* pound sym;
literal → number | symbol | characterConstant | string | tuple;
classToken → (whitespace | comment)* "class" ;
selfToken → (whitespace | comment)* "self" ;
superToken → (whitespace | comment)* "super" ;
outerToken → (whitespace | comment)* "outer" ;
tuple → lcurly [expression (dot expression)* [dot]] rcurly;
block → lbracket [blockParameters] codeBody rbracket;
blockParameters → blockParameter+ vbar;
blockParameter → colon slotDecl;
slotDecl → identifier [type];
codeBody → [temporaries] statements;
temporaries → slotDecls;
slotDecls → seqSlotDecls | simSlotDecls;
returnStatement → hat expression [dot];
statements → returnStatement |
              statementSequence |
              empty;
statementSequence → expression [furtherStatements];
furtherStatements → dot statements;
parenthesizedExpression → lparen expression rparen;
message → keywordMsg | unarySelector | binaryMsg;
binaryMsg → binarySelector unaryExpression;
```

```
    keywordMsg →  (keyword binaryExpression)+;
    primary →  unarySelector |
                literal |
                block |
                parenthesizedExpression;
    unaryExpression → primary unarySelector*;
    binaryExpression →  unaryExpression binaryMsg*;
    keywordExpression →  binaryExpression [keywordMsg];
    implicitKeywordSend → keywordMsg;
    sendExpression → implicitKeywordSend | cascadedMessageExpression;
    expression → [setterKeyword] sendExpression;
    nontrivialUnaryMessages → unarySelector+ binaryMsg*  [keywordMsg];
    nontrivialBinaryMessages → binaryMsg+ [keywordMsg];
    keywordMessages → keywordMsg;
    nonEmptyMessages → nontrivialUnaryMessages |
                       nontrivialBinaryMessages  |
                       keywordMessages;
    cascadeMsg → semicolon (keywordMsg | binaryMsg | unarySelector).
    msgCascade → nonEmptyMessages cascadeMsg*;
    cascadedMessageExpression → primary [msgCascade];
    typeParameterDecl → langleBracket lbracket identifier (comma identifier)* rbracket rangleBracket;
    classDeclaration → (classToken identifier [typeParameterDecl] messagePattern equalSign
                          inheritanceListAndOrBody) |
                        (classToken identifier [typeParameterDecl] empty equalSign
                          inheritanceListAndOrBody);
    inheritanceListAndOrBody → defaultSuperclassAndBody |
                               explicitInheritanceListAndOrBody;
    defaultSuperclassAndBody → classBody;
    explicitInheritanceListAndOrBody → inheritanceClause mixinSpec;
    mixinSpec →  classBody | mixinAppSuffix;
    mixinAppSuffix → (mixinOperator inheritanceClause)+
                      (dot | classBody);
    inheritanceClause → [inheritancePrefix] unarySelector [message];
    inheritancePrefix → outerReceiver |
                        selfToken |
                        superToken;
    outerReceiver → outerToken identifier;
    classBody → classHeader sideDecl [classSideDecl];
    classHeader →  lparen  [classComment]
                     [slotDecls] initExprs  rparen;
    classComment  → [whitespace] comment;
    initExprs → [expression (dot expression)* [dot]];
    sideDecl → lparen (nestedClassDecl | lazySlotDecl | methodDecl)* rparen;
    accessModifier →  ("private" |
```

```
                    "public" |
                    "protected") whitespace;
    nestedClassDecl → [accessModifier] classDeclaration;
    classSideDecl → colon lparen methodDecl* rparen;
    seqSlotDecls → vbar, slotDefs vbar;
    slotDefs → slotDef*;
    slotDef →  [accessModifier] slotDecl
        [((equalSign | assignSlot) expression dot) | dot];
    simSlotDecls → doublevbar slotDefs doublevbar;
    lazyModifier → "lazy" whitespace;
    lazySlotDecl → ((lazyModifier [accessModifier]) | (accessModifier lazyModifier))
                    slotDecl (equalSign |  assignSlot) expression dot;
    methodDecl → [accessModifier] messagePattern equalSign
                lparen codeBody rparen;
    messagePattern:: (unaryMsgPattern |
                    binaryMsgPattern |
                    keywordMsgPattern) [returnType];
    unaryMsgPattern → unarySelector;
    binaryMsgPattern → binarySelector slotDecl;
    keywordMsgPattern → (keyword slotDecl)+;
    compilationUnit →  languageId toplevelClass;
    languageId  →   identifier;
    toplevelClass → classCategory classDeclaration;
    classCategory → [string];
    arg → (whitespace | comment)* "arg";
    for → (whitespace | comment)* "for";
    generic → (whitespace | comment)* "generic";
    inheritedTypeOf → (whitespace | comment)* "inheritedTypeOf";
    is → (whitespace | comment)* "is";
    mssg → (whitespace | comment)* "message";
    of → (whitespace | comment)* "of";
    receiverType → (whitespace | comment)* "receiverType";
    subtypeOf → (whitespace | comment)* "subtypeOf";
    typeArg → (whitespace | comment)* "typeArg";
    where → (whitespace | comment)* "where";
    returnType → hat type;
    type → langleBracket typeExpr rangleBracket;
    typePrimary → identifier [typeArguments];
    typeFactor → typePrimary | blockType | tupleType | parenthesizedTypeExpres-
sion;
    parenthesizedTypeExpression → lparen typeExpr rparen;
    typeTerm → typeFactor identifier*;
    typeExpr → typeTerm [(vbar | semicolon | slash) typeExpr];
    typeArguments → lbracket typeExpr (comma typeExpr)* rbracket;
    tupleType → lcurly [typeExpr (dot typeExpr)*] rcurly;
    blockArgType → colon typeTerm;
```

blockReturnType → typeExpr;
nonEmptyBlockArgList → blockArgType+ [vbar blockReturnType];
blockType → lbracket (nonEmptyBlockArgList | [blockReturnType]) rbracket;
typePattern → langleBracket typeFormal (semicolon typeFormal)* rangleBracket;
typeFormal → where identifier [typeParamConstraint] is inferenceClause;
typeParamConstraint → langleBracket [typeBoundQualifier] typeExpr rangleBracket;
typeBoundQualifier → subtypeOf | inheritedTypeOf;
inferenceClause → receiverType
| (returnType returnTypeInferenceClause)
| typeArgInferenceClause
| (arg number [of msgSelector]);
returnTypeInferenceClause → of msgSelector;
msgSelector → symbolConstant mssg of inferenceClause;
typeArgInferenceClause → typeArg number for generic symbol of inferenceClause;